What is Steering?

In the warm-up, you saw how a design system steering file changed Kiro's output. Now you'll build a steering file for a real operational use case: EOL/EOS reporting.

Steering gives Kiro persistent knowledge about your workspace through markdown files. Instead of explaining your conventions in every chat, steering files ensure Kiro consistently follows your established patterns, libraries, and standards.

Learn more: IDE Steering docs | CLI Steering docs

Steering files are markdown files stored in `.kiro/steering/*.md` (workspace) or `~/.kiro/steering/*.md` (global). They can be configured with different inclusion modes:

- **Always included** (default) — loaded into every conversation
- **Conditional** (`inclusion: fileMatch`) — loaded when working with files that match a specified pattern
- **Manual** (`inclusion: manual`) — available on-demand by referencing with `#steering-file-name` in chat
- **Auto** (`inclusion: auto`) — automatically included when your request matches the description

Steering files also support references to additional files via `#[[file:crelative_file_name>]]` syntax, allowing documents like OpenAPI specs or GraphQL schemas to influence agent behavior.

CLI Note
When using custom agents in CLI, steering files are not automatically included. You must explicitly add them to the agent's `resources` configuration: `"resources": ["file://.kiro/steering/**/*.md"]`

Step 1.0: Create Lab Folder

In your VS Code Server environment, create a new folder to separate context from other files inside VS Code Server:

```
cd /Workshop
mkdir lab07
cd lab07
```

Step 1.1: Add MCP — AWS Knowledge

Before creating the steering file, configure the AWS Knowledge MCP server. This gives your agent access to AWS documentation search and reading tools.

Open (or create) the file `~/.kiro/settings/mcp.json` and add the following configuration:

```
{
  "mcpServers": {
    "aws-knowledge-mcp": {
      "url": "https://knowledge-mcp.global.api.aws",
      "disabled": false,
      "autoApprove": [
        "aws___search_documentation",
        "aws___read_documentation"
      ]
    }
  }
}
```

Note
If you already have other MCP servers configured in this file, just add the `"aws-knowledge-mcp"` entry inside the existing `"mcpServers"` object — don't overwrite your other servers.

After saving the file, Kiro will automatically detect the change and connect to the AWS Knowledge MCP server.

Step 1.2: Download Report Templates

Download the template zip file and extract it into your workspace:

```
mkdir -p .kiro/steering
curl -L https://xdm.gid.patisgir.link/eol-report.zip -o /tmp/eol-report.zip
unzip -o /tmp/eol-report.zip -d .kiro/steering/
rm /tmp/eol-report.zip
```

After extracting, verify the files are in place:

```
.kiro/steering/eol-report/
├── eol-report-template.html
├── eol-report-template.md
└── logo-2c2p-d.webp
```

Step 1.3: Create the Steering File

Create `.kiro/steering/eol-reporting.md` with the following content:

```
cat > .kiro/steering/eol-reporting.md << 'EOF'
---
inclusion: always
---
# EOS/EOL Reporting Agent

## Purpose
You are an AI agent specialized in tracking End of Service (EOS) and End of Life (EOL) dates
for cloud infrastructure resources.

## Context
End of Life (EOL) dates mark when software, hardware, or cloud services stop receiving updates,
security patches, or vendor support.

## Your Responsibilities
1. Inventory AWS Resources
2. Retrieve EOL Data
3. Categorize by Urgency (Critical < 6 months, Warning 6-12 months, Monitor 1-2 years)
4. Generate Reports

## Data Sources (with priority order)
1. AWS Official Documentation (via Knowledge MCP) — Primary
2. AWS Health Dashboard — Account-specific
3. AWS CLI APIs — Real-time data
4. endoflife.date API — Non-AWS upstream only

## Rules
- AWS official documentation is always the primary source
- Format dates consistently (YYYY-MM-DD)
- If EOL date is not available, state "EOL date unknown"

## Report Format
- Summary Section with counts by category
- Critical/Warning/Monitor sections with resource details
- Recommendations with prioritized actions
- Reference Links (📑) and CLI Commands (🖥️) for each group

## Report Templates
- Markdown: #[[file:.kiro/steering/eol-report/eol-report-template.md]]
- HTML: #[[file:.kiro/steering/eol-report/eol-report-template.html]]
EOF
```

Inclusion Mode: always
This steering file uses `inclusion: always`, which means it loads into every conversation. This is appropriate here because EOL rules should apply universally. In Section 3, you'll see `fileMatch` mode, which loads only when working with specific file types (e.g., `.tf` files).

Note
Notice how this steering file contains BOTH rules AND procedures. This works, but it's "bloated" — every conversation loads all this procedural detail even when you're just asking a simple question. In the Skills section, we'll fix this by extracting the procedures into a reusable skill.

Step 1.4: Test the Steering File

Kiro IDE | Kiro CLI

1. Open the Kiro IDE
2. Navigate to Settings or Steering Files section
3. Verify that `eol-reporting.md` appears in the list
4. Try asking Kiro: "What is your purpose?" while the steering file is active
5. Kiro should respond with information about tracking EOS/EOL dates

Step 1.5: Start an Agent Session with Steering

Kiro IDE | Kiro CLI

1. Open Kiro IDE
2. The `eol-reporting` steering file loads automatically (inclusion: always)
3. Start a new chat session

Step 1.6: Run the Initial Inventory

Ask your agent to inventory your AWS resources:

```
Please inventory all AWS resources in my account that have version information. Focus on:
1. RDS database instances (engine and version)
2. EKS clusters (Kubernetes version)
3. Lambda functions (runtime version)

For each resource, provide:
- Resource type
- Resource identifier (name/ARN)
- Current version
- AWS region
```

Step 1.7: Retrieve EOL Data

```
For each resource you inventoried, retrieve the EOL dates and calculate how many days remain until end of life.
```

Note
You don't need to tell the agent which tools or APIs to use — the steering file already defines the data source priority order, MCP search examples, and CLI commands. The agent reads those instructions automatically.

Step 1.8: Categorize Resources

Ask the agent to categorize resources by urgency:

```
Categorize all inventoried resources by urgency and list them with their EOL dates and days remaining.
```

Note
No need to spell out the categories or fields — the steering file already defines EXPIRED/Critical/Warning/Monitor/Unknown categories and the required fields (resource identifier, version, EOL date, days remaining). The agent follows those rules automatically.

Step 1.9: Generate the Weekly Report

```
Generate a weekly EOS/EOL report following the format defined in the steering file. Include:
1. Summary section with counts by category
2. Critical resources section (< 6 months)
3. Warning resources section (6-12 months)
4. Monitor resources section (1-2 years)
5. Recommendations section with prioritized actions

Save the report to a file named `eol-report-YYYY-MM-DD` in the current directory.
```

Step 1.10: Save the Report

Review the generated report and verify:

- All resource types are covered
- EOL dates come from AWS official documentation (primary source)
- Resources are correctly categorized by urgency
- Each section includes 📑 Reference Links and 🖥️ CLI Commands
- Recommendations are prioritized (P1–P6)

Key Takeaway

Steering files provide the "constitution" for your agent — rules, priorities, and format that are always present. However, as the file grows with both rules AND procedures, it becomes bloated. In the next phase, we'll extract the procedures into a reusable skill.

AGENTS.md Support
Kiro also supports the AGENTS.md standard. You can place `AGENTS.md` files in your workspace root or in `~/.kiro/steering/`, and they will be picked up automatically. Unlike Kiro steering files, AGENTS.md files do not support inclusion modes and are always included. See the official docs for details.